
Hosting Platform Decision

KrushiVarva — infrastructure, cost and execution plan for a platform sized to **10,000 users**

Repository	CROPSETU2 · branch main · 1f41862
Date	5 September 2026
Target	10,000 registered users — see §2 for the derivation
Supersedes	The 100,000-user version of this document
Basis	Read from the working tree and docs/performance/BASELINE.md

Headline: Railway, one replica of each service, roughly **\$110 per month all-in**. At this size the architecture you already have is comfortably over-provisioned. Two things still matter and neither is a scaling problem: the database has **no tested restore**, and a handful of correctness defects bite identically at 100 users and at 100,000.

1. Executive summary

RECOMMENDATION

Railway. One replica of each of the four services. Do not plan a migration. At 10,000 registered users the system peaks at roughly 10–30 requests per second, which is around **5–10% of what a single Express replica handles**. There is no capacity problem to solve, and no cost pressure that would justify moving.

The work worth doing is therefore not scaling work. It is **(a)** giving the database a restore path, **(b)** closing correctness defects that do not care how many users you have, and **(c)** a \$90/month media-delivery fix that currently costs more than every server you run combined.

Total cost of ownership

Line	Today	At 10,000 users	Driver
Compute (4 services, 1 replica each)	\$45	\$45	Flat — no replica growth needed
PostgreSQL (managed, with PITR)	\$19	\$19	Flat at this data volume
Redis (persistent)	\$6	\$6	Flat
Media delivery R2 or Bunny	\$2	\$11	~144 GB/month egress
Media delivery on Cloudinary	\$0	\$99	free tier exceeded at ~25 GB
AI providers	\$5	\$15	Assumes cheap-model routing
SMS OTP (MSG91)	\$3	\$12	~5,000 messages/month
Monitoring (Sentry + uptime)	\$0	\$0–26	Free tiers cover this size
Total	~\$80	~\$110	

Modelled from resource shapes read in the code, not measured in production. Arithmetic in sections 6 to 8.

The bill barely moves between "today" and "ten thousand users" because **nothing in the topology has to change**. That is the single most useful fact in this document: your infrastructure decision is already made, and it is correct.

The four things to actually do

#	Action	Effort	Saves / prevents	Why
1	Managed Postgres with PITR	Half a day	Total data loss	The Railway plugin is one container on one volume with no restore you have ever tested. Orders, payments and the credit ledger live there.
2	Close the correctness defects in \$5	2–3 days	Silent failure, double-charges	These are bugs, not capacity limits. Shrinking the user target does not shrink them.
3	Media delivery to R2 or Bunny	1 day	~\$88/month	Cloudinary at this traffic costs more than your entire backend. Largest single cost lever you have.
4	Cloudflare in front of the API	1 hour	~180 ms setup latency	Free. Terminates TLS in India while the origin stays in Singapore.

That is the whole plan. Roughly **four days of work**, after which the hosting question is closed until the user target changes.

2. What "10,000 users" means in load terms

Capacity decisions cannot be made from a registered-user count. The derivation below uses the same ratios the project's own directive applies at 100,000 users, scaled down by a factor of ten.

Quantity	At 100,000 users (previous target)	At 10,000 users (this document)	Ratio used
Registered users	100,000	10,000	given
Daily active users	20,000	2,000	20% of registered
Peak concurrent users	3,000–5,000	300–500	~20% of DAU at peak
Peak API traffic	100–300 RPS	10–30 RPS	proportional
Peak database load	~1,200 q/s	~120–400 q/s	~4–13 queries/request
Concurrent Socket.IO connections	3,000–5,000	300–500	tracks concurrent users
Crop scans per day	~1,000	~100	5% of DAU
Media delivery	~1.44 TB/month	~144 GB/month	30 views/DAU at 80 KB

Confirmed reading: 10,000 is the **total registered user base**, not a daily figure. Every number in this document derives from that. The 20% daily-active ratio is the one carried over from the project's own directive; if real engagement runs hotter — say 40% — the load column doubles and the system is still comfortably inside one replica.

The headroom this buys

backend/src/config/db.js pins connection_limit=12 per replica, and the comment there derives it by Little's law for a **300 RPS** target:

```
fast path  300 req/s × ~4 queries/req = 1200 q/s × ~2 ms      ≈ 2.4 busy connections
slow tail  1% at ~250 ms (geo scans, trigram, admin aggregates) ≈ 3.0 busy connections
fleet ≈ 5.4 busy; ×2 so queueing stays negligible ≈ 11 fleet-wide
```

Your peak is **10–30 RPS, one tenth of that**. The same pool that was sized for four replicas at full load is now serving a single replica at a tenth of the load. Concretely:

- **One Express replica is enough**, with roughly 5–10× headroom before the event loop becomes the constraint.
- **Database connections: 12 of 100**. The budget in the baseline reserved 72 for Express across six replicas; you will use twelve. A connection pooler is not merely unnecessary, it would be solving nothing.
- **300–500 concurrent sockets on one Node process is unremarkable**. Node handles thousands. Socket.IO's Redis adapter exists to coordinate *multiple* replicas; with one replica it is inert, and it already falls back to the in-memory adapter cleanly (server.js:93).
- **Twelve cron schedules on one replica run twelve times, not twelve times N**. The leader-lock multiplication problem needs two or more replicas to exist at all.

3. Platform choice

The shape of the stack, not its size, is what narrows the field. Four properties of the code disqualify most platforms before price is considered.

Constraint	Where in the code	What it rules out
Long-lived WebSockets	<code>server.js:93</code> — Socket.IO with a Redis pub/sub pair	Vercel, Netlify, Cloudflare Workers, bare Lambda. Request-scoped runtimes cannot hold a socket open.
Two queue workers	<code>backend/src/worker.js</code> ; <code>fastapi/railway.json</code> with <code>ROLE=worker</code>	Platforms without a first-class non-HTTP process type.
12 cron schedules	<code>server.js:223-437</code> , <code>utils/leaderLock.js</code>	Scale-to-zero platforms. A sleeping container fires no cron.
Redis holds correctness state	Idempotency keys, AI spend counters, JWT jti denylist, leader locks, BullMQ and Celery brokers	Ephemeral or cache-only Redis tiers. Losing this Redis is not a cache miss — it is duplicate charges and double-spent AI credits.

The viable field, scored at 10,000 users

Platform	Cost at this size	Ops burden	Verdict
Railway chosen	~\$70	Very low	Already configured and working: root Dockerfile, four services, role switch, healthchecks. The only weakness is the database plugin, and that is separable without leaving the platform.
Render	~\$85	Low	Equivalent capability, better native database story, slightly more expensive. No reason to migrate <i>to</i> it once you have put Postgres on Neon.
Fly.io	~\$60	Medium	Cheapest of the three and has a Mumbai region. Their managed Postgres is the weaker part. Worth it only if measured India latency turns out to be the binding constraint — see §7.
Single VPS + Coolify (Hetzner / DO Bangalore)	~\$30	High	Genuinely cheapest, and one 8 GB box would run this entire stack. You take on backups, patching, TLS renewal, monitoring and a single point of failure. Saving \$40/month is not worth an engineer's weekend, at any team size.
AWS Mumbai	\$180+	Very high	Premature Three times the cost and weeks of setup for capacity you would use 5% of.
Vercel / Netlify / Workers	—	—	Disqualified Cannot hold a Socket.IO connection, run Celery, or fire node-cron. Not a cost question.

THERE IS NO EXIT TRIGGER AT THIS SCALE

The 100,000-user version of this document set a threshold for leaving Railway at roughly \$400/month. At 10,000 users the bill lands near **\$70** and stays there. You will not approach that threshold. **Treat the platform question as closed** and stop spending attention on it — that is the correct outcome of asking it.

4. The one thing that does not get easier: the database

HIGHEST-SEVERITY FINDING, AND IT IS NOT A SCALING ISSUE

The Railway PostgreSQL plugin is a **single container attached to a single volume**: no point-in-time recovery, no automatic failover, no read replica, and no restore you have ever exercised.

This database holds orders, payments, the AI credit ledger, KYC records and consent history. A volume failure, a mistaken manual SQL statement, or an accidental service deletion is **unrecoverable**. Ten thousand real users' orders are not less valuable than a hundred thousand.

Three facts already recorded in the repository make the exposure worse:

1. **Schema changes are applied by hand.** `prisma/migrations/` contains 13 directories but only 65 `CREATE TABLE` for 90 models and 23 `CREATE TYPE` for 43 enums. It cannot rebuild a fresh database, so `prisma migrate deploy` is unavailable and changes go through `prisma/manual/*.sql`. Manual SQL against production with no PITR is the most common way small teams lose data.
2. **Two tables are invisible to Prisma.** `ai_scan_diagnoses` and `ai_scan_feedback` are created by FastAPI through `asyncpg` and do not appear in `schema.prisma`. A restore rebuilt from the Prisma schema silently omits them.
3. **There is no CI.** No `.github/workflows` runs the backend tests, the FastAPI tests, or Prisma validation on any push. With hand-applied migrations, CI is the only guard rail there could be.

Where to move it

Option	\$/month	Why it fits	Caveat
Neon preferred	\$19	Point-in-time recovery, and database branching gives you a real throwaway copy per pull request — which is exactly what hand-written migration SQL needs before it touches production.	Confirm the Singapore region is available on the plan you pick.
Supabase	\$25	Managed Postgres with daily backups, PITR as an add-on, and a Mumbai region.	You will use almost none of the surrounding platform.
Railway plugin (today)	\$16	Nothing to do.	Not acceptable once real orders exist. Saves \$3/month against an unbounded loss.

The migration is already unblocked. The boot-time `prisma db push` that held a session-scoped advisory lock was removed in commit `fe95648`, and the `Dockerfile` now starts with `node src/server.js` alone. Nothing in the deploy path fights a managed provider or its pooler. This is a half-day of work, not a project.

5. What does *not* scale down

Reducing the user target removes capacity work. It removes none of the following, because none of these are load-dependent. Several are **more** dangerous at small scale, where there is not enough traffic for a problem to announce itself.

Defect	Reference	Why 10,000 users does not help
No Celery worker liveness signal	PERF-010	FastAPI reports healthy while zero workers are deployed, and every crop scan queues forever. At 100 scans a day this goes unnoticed for <i>longer</i> than it would at 1,000 — there is no traffic spike to make it obvious. Users simply stop getting results.
AI spend cap missed chat and voice	PERF-038	The daily cap was only ever measuring scans. A runaway loop or a prompt-injection costs the same whether you have ten users or ten thousand — the cap is protection against a bug, not against volume.
Credit and spend reservation races	PERF-040, §53	A read-then-write check lets concurrent requests all pass the same limit. It needs two simultaneous requests, not two thousand. One user double-tapping a button is enough.
Payment and stock concurrency	§51, §52	Overselling one item to two buyers is the same defect and the same refund at any scale. The serializable transaction and the unique constraints must hold from the first order.
Unbounded process maps	PERF-039	A long-lived Map keyed by request or IP grows with <i>unique keys seen</i> , not with concurrent users, and a single replica that never restarts accumulates them indefinitely. Small scale makes this worse, not better.
38 failing tests, no CI	PERF-001, §60	Roughly 24 of the 38 are one contract disagreement: tests assert HTTP 422 where <code>middleware/validate.js</code> sends 400. Until that is settled there is no clean regression signal for <i>any</i> change, including the database migration in §4.
Dev OTP fallback in production	MSG91 / DLT	With no MSG91 key the server returns the OTP in the API response instead of sending it. That is a complete authentication bypass if it ever reaches production, and it is entirely independent of user count.

THE REFRAME

Scaling the target from 100,000 to 10,000 users deletes most of the performance backlog. It deletes **none** of this table. If there is one section of this document to act on, it is this one — the platform question above was already answered by the status quo.

What you can now stop worrying about

Explicitly listed so the effort goes somewhere useful. Every item below was justified at 100,000 users and is **premature at 10,000**:

Deferred work	Threshold at which it becomes real
PgBouncer / connection pooler	~6 Express replicas. You will run one. Never needed at this size.
Moving cron off the web tier	Two or more replicas, where the two unlocked schedules start multiplying.
Scoping Socket.IO presence fan-out	A global broadcast is $O(C)$. At $C = 400$ it is free; it becomes real around $C = 5,000$.
Converting the remaining OFFSET pagination	Deep offsets on large tables. Your tables will be small. PERF-019 already recorded these as deliberately not converted.
Auth hot-path cache	One user lookup per request is ~30 q/s at peak here. Postgres does thousands. Cheap to add, but it is not urgent.
Table partitioning, archival, retention tiers	Tens of millions of rows. At 2,000 DAU, <code>mandi_prices</code> is the only table with meaningful independent growth.
Read replicas, multi-region, dedicated search	Not on this roadmap. Postgres trigram search is more than sufficient.

6. Compute cost model

Modelled on Railway's usage pricing shape. **Verify the current published rates before budgeting** — the arithmetic is what matters and survives a price change.

Assumed unit rates: memory $\approx$ \$10 / GB / month
 vCPU $\approx$ \$20 / vCPU / month
 egress $\approx$ \$0.05 / GB

Service	Replicas	RAM	vCPU (avg)	\$/month	Utilisation at peak
Express backend + admin SPA	1	1.0 GB	0.3	16	~10–20%
Node worker (BullMQ)	1	0.5 GB	0.1	7	low
FastAPI (<code>--workers 2</code>)	1	0.5 GB	0.15	8	low
Celery worker (concurrency 2)	1	0.5 GB	0.15	8	~100 scans/day
PostgreSQL — Neon	—	—	—	19	~120–400 q/s
Redis (persistent)	1	0.4 GB	0.1	6	low
Egress (~60 GB, media excluded)	—	—	—	3	—
Total compute + data		2.9 GB	0.8	\$67	

Railway's plan fee (Hobby \$5, Pro \$20) includes an equivalent usage credit, so the effective bill is close to the usage total.

Two consolidations available, and why to skip one

- **Drop the standalone Node worker.** The Express replica already runs an in-process BullMQ consumer at `QUEUE_CONCURRENCY=5`, so the separate `worker.js` service is optional. Saves **\$7/month**. **Keep it anyway** — it is the only isolation between a slow notification job and the HTTP event loop, and \$7 is not a reason to give that up. Verify that only one of the two is consuming the queue in production, so jobs are not picked up by both.
- **Reduce FastAPI to one uvicorn worker.** `WEB_CONCURRENCY` defaults to 2. At this traffic one is genuinely sufficient and it halves the FastAPI memory footprint and its `asynpg` pool usage. Saves ~\$3/month and some database connections. Low risk, do it if you are tuning.

Growth headroom

The useful question is not what 10,000 users costs, but at what point the bill changes shape:

Milestone	Compute + data	What changes
10,000 registered target	\$67	Nothing. One replica each.
25,000 registered	~\$90	Second Express replica for redundancy rather than capacity. Cron leader locks start to matter.
50,000 registered	~\$160	Two to three Express replicas, larger Postgres. Auth cache and cron separation become worth doing.
100,000 registered	~\$640	Five or six replicas, a connection pooler, and the platform question genuinely reopens.

Between here and 50,000 registered users, **the infrastructure does not need a decision**. It needs one extra replica and a bigger database, both of which are dashboard settings.

7. Region and latency

Railway's nearest region is Singapore; there is no India region. The users are farmers on low-end Android devices on 2G, 3G and weak LTE. Both facts are fixed, and the gap between them is closed by a CDN rather than by a migration.

Path	RTT	Cold connection setup	Cost to obtain
India → Singapore, direct	55–75 ms	~200–225 ms	current state
India → Cloudflare India PoP	10–25 ms	~40–75 ms	free, one hour
India → Mumbai origin	15–30 ms	~55–90 ms	full platform migration

Cold setup approximated as three round trips for TCP plus TLS 1.3. Real figures vary widely on rural mobile networks.

The middle row is the one to buy. Proxying through Cloudflare terminates TLS at an India point of presence, so the phone's expensive handshake happens locally and the Singapore leg rides a warm, pooled connection. It recovers most of the penalty for nothing, and it leaves the origin exactly where it is.

The residual 40 ms between Cloudflare and a Mumbai origin is not worth a migration. The baseline records GET /agristore/cart issuing **13 database queries** and POST /agristore/orders/confirm issuing **25–35**. Against a 500 ms p95 target, application and query time dominate by an order of magnitude. Fix the cart before fixing the continent.

Enabling it, in order

1. **Proxy the API domain** through Cloudflare (orange cloud). Free plan.
2. **Confirm WebSockets are enabled** in the dashboard. If they are not, Socket.IO silently degrades to HTTP long-polling, which multiplies your request count and defeats the point of having sockets at all.
3. **Do not cache /api/v1/* by default**. Authenticated JSON must never be shared between users. Add cache rules only for genuinely public, non-personalised reads.
4. **Check proxy-aware rate limiting before you switch it on**. Rate limits key on client IP; behind a proxy every request arrives from a Cloudflare address unless `trust proxy` is set and the forwarded header is read. Get this wrong and the limiter either buckets the whole country together or trusts a spoofable header.
5. **Serve media through the CDN**, not the origin — see §8.

8. Media: the line item larger than your servers

AT 10,000 USERS, IMAGE DELIVERY COSTS MORE THAN EVERY SERVER YOU RUN

Modelled at **~\$99/month on Cloudinary** against **~\$67/month** for the entire backend, database and cache combined. The identical traffic on Cloudflare R2 or Bunny is **\$2–11/month**.

2,000 DAU × ~30 image views per active day × ~80 KB per card-sized transform
= 4.8 GB / day
= ~144 GB / month of delivery (Cloudinary's free allowance ≈ 25 GB)

Provider	144 GB delivery	Storage	Notes
Cloudinary (current)	\$99	included	Free tier is exceeded around 25 GB, which lands somewhere near 2,000 daily actives. The next tier is a step, not a slope — you pay \$99 the month you cross it.
Cloudflare R2 cheapest	\$0	~\$1	Zero egress through Cloudflare is the entire reason to pick it. Generate sizes on upload with sharp in the Node worker you already run.
Bunny.net	~\$1.50	~\$1	Plus roughly \$10/month for the image optimiser if you want on-the-fly transforms rather than writing them. Simplest drop-in.

Saving: about \$88 per month, or \$1,050 a year, for roughly a day of work — and it grows with every user you add. This is the highest return per hour available anywhere in this document.

Why it is tractable

- The database stores `publicId` and URL, never binary. Changing delivery host is a URL-construction change, not a schema migration.
- `backend/src/config/cloudinary.js` centralises upload handling behind one module, with limits already set per surface: 15 MB general, 5 MB avatar, 100 MB video.
- The upload path has been worked on recently — PERF-025 (uploads cut off while Cloudinary was still receiving) and PERF-035 (a process buffering unbounded 100 MB videos) are both recorded — so it is understood code, not archaeology.

Order of operations

1. **Move delivery first, keep Cloudinary for transformation** if you want the smallest possible first step. Most of the saving is in egress.
2. **Generate four sizes on upload** (thumbnail, card, detail, original) in the Node worker. Fixed cost at write time beats metered transforms at read time.
3. **Audit the mobile list screens** for a 3000 px original rendered into a 120 px card. Do this *before* migrating, or you will simply pay for the same mistake more cheaply.
4. **Signed direct upload** — the client requests a signed URL, uploads straight to object storage, and posts back only a media ID. This also removes the base64-through-JSON path that the AI scan currently uses,

cutting Node memory and GC pressure in the same change.

9. AI, SMS and the rest

Item	At 10k users	Note
AI providers	\$15–130	~100 scans and ~1,000 chat turns per day. The 10× spread is entirely model routing , not volume: cheap-model routing lands near \$15, a large-model or ensemble default near \$130. <code>docs/WI11-model-routing.md</code> already exists — what matters is that it is enforced, and that the daily spend cap covers chat and voice, not only scans.
SMS OTP (MSG91)	\$12	~5,000 messages/month at roughly ₹0.15–0.25 each. Blocked on DLT approval ; the dev OTP fallback must not reach production — see §5.
Expo EAS	\$0	Free tier covers this build volume. Push delivery itself is free.
Monitoring	\$0–26	Sentry's free tier is adequate at this size; add an uptime checker. Non-optional given \$5 — most of those defects fail silently.
Weather / mandi APIs	\$0	Open-Meteo and data.gov.in are free. Cache aggressively and you will not reach a paid tier.
Razorpay	~2% of GMV	Transaction cost rather than infrastructure, but it belongs in the model.

10. Execution plan

Week 1 — before real users

#	Action	Effort	Why first
1	Move Postgres to Neon	Half a day	Removes the only unrecoverable failure mode. Everything else on this list is reversible.
2	Restore into a scratch database and verify it	1 hour	An untested backup is not a backup. Same day, not later. Check that the two FastAPI-owned tables came across.
3	Cloudflare in front of the API	1 hour	Free, and the largest latency win available. Verify WebSockets and proxy-aware rate limiting.
4	Alert on Celery worker heartbeat	2 hours	Crop scans queue forever today with nothing to notice it. At 100 scans/day nothing else will surface it.
5	Confirm the dev OTP fallback cannot fire in production	30 min	Authentication bypass. One environment variable stands between you and it.

Week 2 to 3 — correctness

1. **Settle the 400-vs-422 contract** and get the test suite green. Roughly 24 of the 38 failures are that one disagreement. Until it is fixed nothing else has a regression signal.
2. **Add CI** that runs the backend suite with `--runInBand`, the FastAPI tests, and `prisma validate`. Parallel jest is not a usable signal — the suites share one database and deadlock.
3. **Extend the AI spend cap to chat and voice**, and make every reservation atomic.
4. **Bound the fallback maps** — max size or TTL, so a single long-lived replica cannot leak.
5. **Verify payment and stock concurrency** holds under two simultaneous requests. Two, not two thousand.

Week 4 — cost and polish

1. **Migrate media delivery to R2 or Bunny**. ~\$88/month, roughly a day.
2. **Audit mobile list screens** for oversized images before or alongside that migration.
3. **Pin AI model routing** to the cheap tier by default and confirm the cap is enforced.
4. **Load test at 50 and 100 concurrent** — well above your real peak — and record RPS, p95, p99, database connections and queue depth. This is a one-afternoon sanity check, not the staged campaign a 100,000-user target would need. Mock the AI providers so the test does not spend real money.

AFTER THAT, STOP

There is no phase two of hosting work at 10,000 users. The next infrastructure decision is a second Express replica somewhere around 25,000 registered users, and it is a dashboard setting. Point the engineering effort at the product instead — that is what the headroom in \$2 is for.

11. What this document does not know

- **No production telemetry exists.** Every cost and load figure is modelled from resource shapes read in the code, not observed. The first real bill will correct them.
- **The 20% daily-active ratio is inherited, not measured.** 10,000 registered is confirmed; the 2,000 DAU that follows is an assumption carried from the project directive. Real engagement could be half or double that, which moves peak RPS between ~5 and ~60 — all of it inside one replica, so the recommendation is insensitive to being wrong here.
- **Engagement drives the media model.** Thirty image views per active day is an assumption. At ten, Cloudinary's free tier might still hold; at a hundred, the media saving is three times larger than stated.
- **Provider prices and regions change.** Confirm Railway's current rates, Neon's Singapore availability, and Cloudinary's current free allowance before acting on a specific number.
- **The 38 failing tests** mean there is no clean regression signal today, which makes every item in \$10 riskier than it needs to be. That is why they are in week two.

One-line summary. Stay on Railway with one replica of each service — roughly \$110/month all-in, and the platform question is closed. Spend the saved effort on a database you can restore, the correctness defects in \$5, and moving image delivery off Cloudinary.